

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Learning-Based Inverse Kinematics: A Quantitative Comparison

Emanuel André Medina Araujo



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Learning-Based Inverse Kinematics: A Quantitative Comparison

Lernbasierte Inverse Kinematik: Ein Quantitativer Vergleich

Author:	Emanuel André Medina Araujo
Supervisor:	Prof. Dr.-Ing. Matthias Althoff
Advisor:	Jonathan Külz
Submission Date:	01.12.2024

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 01.12.2024 Emanuel André Medina Araujo

Acknowledgments

First and foremost, the author would like to thank my supervisor, Jonathan Külz, for his guidance and insightful feedback.

Additionally, we would like to thank the faculty of Cyber-Physical Systems at TUM School of Computation, Information, and Technology for the provided resources.

Abstract

A fundamental issue in robotics is solving a robot's Inverse Kinematics (IK). It is essential for calculating and planning any robot's movements.

This thesis will investigate different learning-based approaches for the IK on manipulators with two or three degrees of freedom (DOF). Once trained, the model can solve the Inverse Kinematics (IK) for every manipulator with the same number of DOF.

We will compare the angles by predicting them directly and via a beta, normal, and two-dimensional mixture of Gaussian distribution. Furthermore, we will evaluate the impact of three network architectures: multilayer perceptron (MLP), Long Short-Term Memory (LSTM), a hybrid of MLP and LSTM, and Proximal Policy Optimization (PPO).

Our findings include using an MLP network to predict the angles directly for the best performance. Secondly, PPO performs the worst of all approaches, and we deduce a non-trivial connection between the network's complexity and the model's performance. Therefore, we recommend that roboticists keep it simple using the MLP network to solve the IK for the best results.

Contents

Acknowledgments **iii**

Abstract **iv**

1. Introduction **1**
1.1. Contributions 2

2. Background **3**
2.1. Forward Kinematics 3
2.1.1. Homogeneous Transformation 3
2.1.2. DH Parameters 3
2.2. Inverse Kinematics 4
2.3. Reinforcement Learning 5
2.3.1. Stochastic vs Deterministic Policy 6
2.3.2. Proximal Policy Optimization 6
2.3.3. Long Short-Term Memory Networks 7

3. Related Work **9**
3.1. Analytical Approaches 9
3.1.1. Conditions for Existence of a Closed-Form Solution 9
3.1.2. Geometric Decomposition 10
3.1.3. Subproblem-Based Solutions 10
3.1.4. Algebraic Methods 10
3.2. Numerical Approaches 11
3.2.1. Jacobian Methods 11
3.2.2. Heuristic and Meta-Heuristic Methods 13
3.3. Learning-based Approaches 13
3.3.1. Regression-Based Approaches 13
3.3.2. Generative Models 13
3.3.3. Graph Neural Networks 14
3.3.4. Bayesian and Variational Methods 14
3.3.5. Hypernetworks and invertible networks 14

4. Method **15**
4.1. Problem Definition . . . 15
4.2. Data Generation . . . 16
4.3. Neural Network Architectures . . . 16
4.3.1. Deterministic Policies . . . 16
4.3.2. Stochastic policies . . . 17
4.3.3. Recurrent policies . . . 17
4.3.4. Hybrid policies . . . 18
4.4. Implementations . . . 18
4.4.1. Predicting joint angles deterministically . . . 18
4.4.2. Predicting joint angles stochastically . . . 18
4.4.3. Predicting a normal distribution of joint angles using a MLP . . 18
4.4.4. Reparameterization Trick . . . 19
4.4.5. Predicting a normal distribution of joint angles using an LSTM . 19
4.4.6. Predicting a beta distribution of joint angles using a MLP 19
4.4.7. Predicting a Gaussian mixed distribution of joint angles using an MLP . . . 20
4.4.8. Predicting a Gaussian mixed distribution of joint angles using an LSTM + MLP . . . 20
4.4.9. Predicting a Gaussian mixed distribution of joint angles using an LSTM . . . 20

5. Experiments **21**
5.1. Setup . . . 21
5.2. Results . . . 22
5.2.1. The simplest implementation performs the best for two and three DOF problems . . . 22
5.2.2. MLP and LSTM networks perform better and faster than PPO . 24
5.2.3. Non-trivial connection between complexity and performance . . 25

6. Conclusion and Outlook **29**
6.1. Conclusion . . . 29
6.2. Outlook . . . 29

A. Appendix **30**
A.1. Visualization . . . 30

Abbreviations **34**

Bibliography **35**

1. Introduction

In recent years, the field of robotics has seen significant advancements. The advancements track to the increasing demand for robots in various industries, such as manufacturing [28], healthcare [31], and agriculture [22]. The drastically increased capabilities in terms of computational power and various publications have enabled robots to perform previously considered impossible tasks. For instance, in the 90s, robots were rare in surgery rooms [27], but saving lives they are an indispensable technique today [32].

However, the complexity of the desired tasks has also risen, and robots are now required to navigate complex environments [42] and interact with objects and humans [44] more sophisticated, which resulted in the development of robots with redundant DOF [14].

Redundant robots refer to robotic systems that can reach a specific position with more than one configuration of links [12]. This additional DOF allows the robot to accomplish the task in multiple ways, providing more flexibility and adaptability. Nonetheless, this flexibility also introduces challenges, especially when dealing with the manipulator's kinematic model.

Forward and inverse Kinematics are essential problems that need solutions for the use cases mentioned above regarding the kinematic model. The Forward Kinematics (FK) problem is determining the position and orientation of the end-effector given the joint angles of a manipulator. While the FK is solvable [48], the inverse problem, the IK, is not. It consists of determining the joint angles of a manipulator given a desired end-effector position and orientation. Depending on the task, this can result in no solution, one solution, multiple solutions, or even an infinite number of solutions [48].

In general, three main approaches can solve the IK problem: analytical [48], numerical [4], and learning-based [7] methods as well as hybrids which we will discuss in more detail in chapter 3.

This thesis will focus on the learning-based approaches. More into detail, we will concentrate on Reinforcement Learning (RL) approaches, which we will compare and evaluate. Being a quantitative comparison of the learning algorithms, we will restrict ourselves to the IK for a two DOF and three DOF robot manipulator to prevent further complexity of novel and complex manipulators. That restriction enables us to focus on the base performance of the algorithms.

Concerning the RL approaches, we will evaluate the three most common algorithms: MLP, PPO, and LSTM. We will also consider the hybrid of MLP and LSTM networks. As output, we will consider direct angle prediction and predicting a distribution, which we will use to sample an angle. We will consider a normal distribution, a beta distribution, and a two-dimensional mixed Gaussian distribution.

1.1. Contributions

Our main contribution is comparing seven learning-based approaches to solve the IK for any two DOF and three DOF robots once trained. We will analyze the training time and accuracy of the prediction. The comparison will include accuracy, training time, network architecture, used angle prediction method, and prediction run time. The seven approaches are:

- Direct angle prediction using a MLP network
- Direct angle prediction using PPO
- Angle via normal distribution prediction using a MLP network
- Angle via normal distribution prediction using a LSTM network
- Angle via beta distribution prediction using a MLP network
- Angle via mixed Gaussian distribution prediction using a MLP network
- Angle via Gaussian mixed distribution prediction using a combination of a LSTM and a MLP network
- Two angles via Gaussian mixed distribution prediction using LSTM network

Our findings are that the simplest MLP network predicting the angles directly performs the best. Second, we show that the Actor-Critic-Architecture of PPO performs worse than the MLP approach in training time and accuracy. Lastly, we deduce a non-trivial connection between the network's complexity and the model's performance.

This thesis is structured as follows. Chapter 2 provides a basic introduction to the field of Inverse Kinematics and Reinforcement Learning. Chapter 3 discusses an overview of related work in the field of IK and corresponding approaches to solving it. Chapter 4 describes our methodology used for the experiments and the underlying implementations. In chapter 5, we present the experiments conducted to evaluate the performance of the proposed approaches and discuss the findings. Finally, chapter 6 concludes the thesis and outlines future work.

2. Background

In general, robotic arms consisting of $n \in \mathbb{N}_+$ joints have an according joint space of $\mathbb{Q} \subset \mathbb{R}^n$. However, in this thesis, we will restrain ourselves to $n \in \{2, 3\}$ to concentrate on the comparison of algorithms.

2.1. Forward Kinematics

To obtain the FK of a manipulator, we first need to assign a fixed reference frame to each joint to get the position and orientation of each joint to a global reference frame. The global frame is usually equal to the frame on the first joint of the manipulator. In this thesis, we will also always attach our global frame to the first joint. [48] proposed the homogeneous transformation to represent frame transformations.

2.1.1. Homogeneous Transformation

$$ {}^{l}T_m = \begin{pmatrix} {}^{l}R_m & {}^{l}t_m \\ 0^T & 1 \end{pmatrix} \tag{2.1} $$

This is the homogeneous transformation matrix. The rotation ${}^{l}R_m$ combined with the translation ${}^{l}t_m$ represents the transformation from frame l to frame m.

2.1.2. DH Parameters

Having introduced a possibility to describe the transformation between two frames, we can now calculate the transformation between two consecutive frames of a manipulator. The Denavit-Hartenberg (DH) parameters introduced in [17] are a common way to describe the transformation between two consecutive link frames. The DH parameters consist of four parameters and are defined as follows: (1) α the angle between z_{j-1} and z_j with y_{j-1} as rotation axis, (2) a, the link length (3) θ, the angle between x_{j-1} and x_j with z_{j-1} as rotation axis (4) d, the offset in direction of z_{j-1}.

This thesis will use the DH parameter to describe a manipulator. Since we are only interested in the position of two and three DOF manipulators, we will only consider a and θ. [35] surveyed the differences between the conventions.

Some have modified the DH parameters by adjusting the order of transformations and altering the location where frame F_{j-1} is attached to axis $j-1$ instead of axis j. The frame F_1 is also attached to axis j rather than axis $j+1$. Modified DH (MDH) parameters is the name of this alternative [36].

When concatenating all transformations from the base frame to the end effector frame, we obtain the forward kinematics with:

$$^0T_{eef} = {}^0T_1 \, {}^1T_2 \, ... \, {}^{eef-1}T_{eef} \tag{2.2}$$

Another common perspective on forward kinematics is to differentiate for the time of the forward position kinematics [48]:

$$^l v_m = J(q)\dot{q} \tag{2.3}$$

where $J(q)$ is a function for joint angles q and called the Jacobian matrix. More into detail, the Jacobian matrix is $J(q) = [J_1 J_2 ... J_N]$ where J_i is the i-th column of the Jacobian matrix and N is the number of joints. The Jacobian matrix relates to the joint velocities $\dot{q}$ to the end effector velocity $^l v_m$ in any frame l. We will return to the Jacobian matrix in the next section 2.2 and in chapter 3.

We can now define the forward kinematics function $f_{FK} : \mathbb{Q} \rightarrow SE(3)$ which returns the end effector position $p_{eef} \in SE(3)$, where $SE(3)$ is the set of all reachable End Effector (EEF) positions, for a given joint configuration $q_{eef} \in \mathbb{Q}$. $SE(3)$ is also called the workspace of the robot arm.

2.2. Inverse Kinematics

When inverting the forward kinematics function, we obtain the inverse kinematics function $f_{IK} : SE(3) \rightarrow \mathbb{Q}$ which returns the set of joint configurations $q_{eef} \in \mathbb{Q}$ for a given end effector position and orientation $p_{eef} \in SE(3)$. Since this function is not necessarily injective, we can obtain infinite, multiple, one, or no solutions for a given end effector position.

As mentioned in the previous section, the Jacobian matrix relates the joint velocities to the end effector velocity. The inverse kinematics problem can be solved when the Jacobian matrix is squared by inverting the Jacobian matrix and multiplying it with the end effector velocity.

$$\dot{q} = J(q)^{-1} {}^l v_m \tag{2.4}$$

However, the Jacobian matrix may become singular on specific tasks, leading to non-invertible configurations where small changes in joint angles produce significant or

no changes in the end-effector position. We now want to find a solution for the inverse kinematics problem for any given manipulator and any given end effector position.

2.3. Reinforcement Learning

This thesis will use reinforcement learning to solve the inverse kinematics problem. Even though training a RL model can be computationally expensive, the trained model can solve the IK problem in real-time and can be easily adapted to new robots, as we will see in chapter 5.

Reinforcement learning is a type of machine learning where an agent wants to achieve a specific goal by learning how to interact with a given environment. The agent learns by interacting with the environment and receiving feedback through penalties [56].

The agent interacts with the environment by taking actions a_t at time t based on the current state s_t and receives a penalty l_t and the next state s_{t+1}. That interaction is called a policy $a_t = \mu(s_t)$ that maps a state to an action. If the policy is stochastic, its definition is: $a_t \sim \pi(\cdot|s_t)$ The policy is learned by minimizing the cumulative loss over time. The loss function L is defined as [2]:

$$l_t = L(s_t, a_t, s_{t+1}) \tag{2.5}$$

where s_t is the environment state at time t, a_t is the action at time t and s_{t+1} is the state at time $t+1$. A sequence of states and actions is called trajectory [2]:

$$\tau = (s_0, a_0, s_1, a_1, s_2, a_2, ...s_T) \tag{2.6}$$

The goal of the agent is to minimize the cumulative loss over time by learning a policy that maps states to actions [2]:

$$L = \sum_{t=0}^{T} l_t \tag{2.7}$$

where T is the number of time steps. To receive the optimal policy π^* the agent needs to maximize the expected return defined as [2]:

$$R(\pi) = \int_{\tau} p(\tau|\pi) L(\tau) = \mathbb{E}_{\tau\sim\pi}[L(\tau)] \tag{2.8}$$

where $p(\tau|\pi)$ is the probability of a T-step long trajectory given by [2]:

$$p(\tau|\pi) = p_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1}|s_t, a_t)\pi(a_t|s_t) \tag{2.9}$$

where $p_0(s_0)$ is the initial state distribution and $P(s_{t+1}|s_t, a_t)$ is the transition probability.

Further a value function $V(s)$ is defined as the expected return when starting in state s and following the policy π [2]:

$$V^{\pi}(s) = \mathbb{E}_{\tau\sim\pi}[R(\tau)|s_0 = s] \tag{2.10}$$

This function is often also called critic in the context of reinforcement learning. The value function frequently evaluates the policy π and adapts it accordingly. We will discuss our application of those concepts in chapter 4. For a more detailed introduction to reinforcement learning, we refer to [43, 50].

2.3.1. Stochastic vs Deterministic Policy

Using a deterministic policy, we will always receive the same action for a given state. In contrast, the stochastic policy receives a probability distribution over the actions of a given state. In 5, we will compare the performance of a deterministic policy to a stochastic policy in the context of IK.

2.3.2. Proximal Policy Optimization

Proximal Policy Optimization (PPO), introduced by Schulman et al. [43], is a reinforcement learning algorithm designed to achieve efficient and stable policy updates. PPO builds on policy gradient methods and trust-region optimization principles but simplifies implementation and improves sample efficiency.

PPO optimizes the surrogate objective function derived from policy gradient methods while ensuring that policy updates do not deviate significantly from the previous policy. The key idea in PPO is to restrict the policy update using a clipped objective function:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{2.11}$$

where$r_t(\theta)$ is given by:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{2.12}$$

and represents the probability ratio between the updated and old policies. While $\hat{A}_t$ estimates the advantage function, which indicates how favorable an action is compared to the policy's expectation, ϵ is a hyperparameter defining the clipping range. The clipping term ensures that $r_t(\theta)$ remains within $[1-\epsilon, 1+\epsilon]$. The surrogate objective takes the minimum between the clipped and unclipped values, penalizing large policy updates.

Additionally, [43] introduces an adaptive KL Penalty Coefficient, which may be used instead of or in addition to the surrogate objective. However, Schulman et al. [43] showed that the clipped objective performs worse than the clipped surrogate objective.

PPO consists of an actor-critic framework where the policy (actor) and the value function (critic) are optimized together. The combined objective used in PPO is:

$$L_{\text{PPO}}(\theta) = \mathbb{E}_t \left[L_{\text{CLIP}}(\theta) - c_1 L_{\text{VF}}(\theta) + c_2 S[\pi_\theta](s_t) \right] \tag{2.13}$$

Here, $L_{\text{VF}}(\theta)$ is $(V_\theta(s_t) - V_{\text{target},t})^2$ which represents a squarred error loss, where $V_\theta(s_t)$ is the predicted value of state s_t, and $V_{\text{target},t}$ is the target value. The term $S[\pi_\theta](s_t)$ represents an entropy bonus added to encourage exploration. The coefficients c_1 and c_2 determine the relative weight of the value function loss and the entropy bonus.

The PPO algorithm collects batches of trajectories from the current policy and optimizes the surrogate objective using stochastic gradient descent over multiple epochs.

In this thesis, we will use PPO will to train policies for solving the inverse kinematics problem, as explained in Chapter 5.

2.3.3. Long Short-Term Memory Networks

LSTM networks initially introduced in [26] by Hochreiter et al. are a type of recurrent neural network (RNN) that can learn long-term dependencies in sequential data.

An LSTM network consists of recurrently connected memory cells. Each cell has a forget gate, an input gate, and an output gate that controls the information retention, update, and output processes. The LSTM also includes a cell state vector C_t, which acts as long-term memory, and a hidden state vector H_t, representing short-term memory.

At time step t, an LSTM unit processes three inputs: the current input vector X_t, the previous hidden state H_{t-1}, and the previous cell state C_{t-1}. For initialization, usually $H_0 = 0$ and $C_0 = 0$ will be used. First, the forget gate determines which information from C_{t-1} should be kept and is calculated as:

$$f_t = \sigma(W_f \cdot H_{t-1} + X_t \cdot b_f) \tag{2.14}$$

where W_f and b_f are the weights for the input, the hidden state X_t is the input vector, and σ denotes the sigmoid activation function.

Which parts of a new candidate memory vector integrate into the cell state decides the input gate. The input gate and candidate memory computations are:

$$i_t = \sigma(W_i \cdot H_{t-1} + X_t \cdot + b_i) \tag{2.15}$$

$$\tilde{C}_t = \tanh(W_c \cdot H_{t-1} + X_t \cdot + b_c) \tag{2.16}$$

where i_t is the input gate output and $\tilde{C}_t$ is the new information. The updated cell state C_t is obtained by combining the forget gate's scaling of C_{t-1} and the input gate's scaling of $\tilde{C}_t$:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \tag{2.17}$$

Finally, the output gate determines the next hidden state H_t, which serves as both the short-term memory and the output of the LSTM unit. To receive the hidden state, we calculate:

$$o_t = \sigma(W_o \cdot H_{t-1} + X_t \cdot b_o) \tag{2.18}$$

$$H_t = o_t \cdot \tanh(C_t) \tag{2.19}$$

Here, o_t is the output gate's scaling factor.

We will use LSTM networks in this thesis to predict the joint angles of a robot arm based on the end effector position. We will discuss the LSTM architecture application in more detail in Chapter 4.

In the next chapter (3), we will discuss different approaches to solving inverse kinematics.

3. Related Work

We aim to solve the IK for all robot arms. We can not always know beforehand if this is possible. This chapter will overview the three main method categories that solve the IK problem: analytical, numerical, and learning-based methods.

3.1. Analytical Approaches

Analytical methods aim to compute closed-form solutions to the IK by directly inverting the forward kinematics equations 2.3 and therefore receiving 2.4. However, that equation is only solvable when J is squared.

Requiring extensive symbolic or geometric manipulation tailored to specific manipulator configurations, they are highly efficient during runtime once solved. [19] and [40] show that the IK can be obtained in the dimension of microseconds, making them suitable for real-time applications. Often, the time to receive an analytical solution is longer than the time to solve the IK numerically based on the fact that it has to be done individually for every manipulator.

Researchers have proposed several methods to solve various classes of kinematic manipulators such as in [1], IKFast introduced in [19], and IKBT presented in [61]. But they all face problems when dealing with high degrees of freedom robots. Tools like IKFast and IKBT utilize symbolic manipulation or geometric decomposition to precompute IK solutions. These solutions are encoded in optimized solver functions.

3.1.1. Conditions for Existence of a Closed-Form Solution

Determining whether a closed-form solution to the IK problem exists is crucial for developing efficient and robust algorithms. While we can often derive solutions for six-degree-of-freedom (DOF) manipulators, higher DOF configurations typically introduce redundancy, leading to infinite possible solutions unless additional constraints are applied [48].

Pieper established two widely acknowledged sufficient conditions for the existence of a closed-form IK solution [39]. (1) At least three revolute axes side by side have to intersect. (2) At least three joints axes side by side are parallel. These conditions have proven foundational for many analytical approaches, especially in designing industrial

manipulators. However, concerns remain about their completeness in addressing unique or unconventional manipulator configurations [45].

An alternative sufficient condition, independent of joint axis intersections, has been proposed by Chen and Gao [11]: The total number of revolute or prismatic joints is three or fewer.

A closed-form solution additionally requires non-degeneracy [61]. Non-degeneracy is when no joint's movement can be expressed as a linear combination of other joints, effectively reducing the manipulator's independent degrees of freedom.

Identifying these conditions before solving the IK problem can save significant computation time. Alternative approaches, such as numerical or learning-based methods, must be considered for manipulators that fail to meet these conditions.

In the following part, we elaborate on analytical techniques, categorized into geometric decomposition, subproblem-based methods, and algebraic formulations.

3.1.2. Geometric Decomposition

A common simplification technique is decomposing the IK problem into smaller subproblems based on the geometric properties of the manipulator [37]. For instance, many manipulators, such as those with spherical wrists, allow separation of the IK into position and orientation problems. This decomposition reduces the complexity of solving, for example, a six degrees of freedom (DOF) manipulator by splitting the problem into two 3-DOF tasks for which solutions may be known [60]. One example of this is simply locking a joint to simplify the problem. This approach is also done in IKFast [19]. Important here is that the correct redundant joint is locked to avoid vast changes in the robot's workspace.

3.1.3. Subproblem-Based Solutions

A subproblems approach like [19] or [11] offers a modular approach by solving predefined geometric tasks. The Paden-Kahan subproblems [59] are widely used and accelerate the derivation of IK for complex manipulators by combining solutions to canonical problems. These subproblems are particularly useful for modular robot designs.

3.1.4. Algebraic Methods

For manipulators lacking geometric simplicity, algebraic methods rely on solving polynomial equations arising from the forward kinematics. There is a vast array of possibilities, but a common approach incorporates the trigonometric identities involving

joint angles θ_i which can extend the work of [25] and [39]. The pair of equations:

$$a\cos(\theta) + b\sin(\theta) + c = 0,\ a\cos(\theta) - b\sin(\theta) + d = 0 \tag{3.1}$$

where a, b, c and d are constants can be rearranged for θ resulting in:

$$\theta = 2\,\mathrm{Atan2}\left(-a \cdot d - b \cdot c, b \cdot d - a \cdot c\right). \tag{3.2}$$

Interestingly, Atan2 enables the calculation of the full range of angles by placing it in the correct quadrant. We will use this property later in 4.3.1.

Analytical approaches remain invaluable for solving IK in structured environments where speed and accuracy are paramount. Nonetheless, deriving analytical IK requires significant manual intervention or computational resources during setup, limiting their adaptability to novel or varying robot designs.

3.2. Numerical Approaches

Those drawbacks have led to the development of numerical methods [6, 48, 55]. These methods use iterative algorithms to solve the IK. They have shown better results with robots with higher DOF and are easier to adapt to new robots. However, they lack computational efficiency and numerical stability, making them unsuitable for real-time applications. They typically provide only one solution per iteration, which might be far from optimal.

Similarly to the IK solving methods, numerical methods are typically divided into Jacobian, heuristic, and meta-heuristic methods.

3.2.1. Jacobian Methods

Researchers widely use Jacobian methods for solving the IK problem. These methods calculate the Jacobian matrix, which represents the relationship between the joint velocities $\dot{q}$ and the end-effector velocities ${}^{l}v_m$ introduced in 2.4. As already mentioned, this is solvable when the Jacobian matrix is squared. For non-square Jacobians, several numerical methods are available.

Jacobian Transpose

The Jacobian transpose method introduced in [57] and computes the transpose of the Jacobian matrix to find the change in joint angles:

$$\Delta q = \alpha J^T \vec{e} \tag{3.3}$$

where α is a scaling factor, and $\vec{e}$ is the desired change in EEF position. This method has a lower computational cost as it avoids calculating the inverse but may not always provide the optimal solution, particularly with multiple end-effectors [10].

Jacobian Pseudo Inverse

The Jacobian pseudo-inverse [10], also known as the Moore-Penrose inverse, solves non-square Jacobians. It is given by:

$$J^{+} = J^{T}(JJ^{T})^{-1} \tag{3.4}$$

We compute the change in joint angles as follows:

$$\Delta q = J^{+}\vec{e} \tag{3.5}$$

Though it can handle singularities better than the transpose method, it is sensitive to near-singularities.

Singular Value Decomposition (SVD)

SVD decomposes the Jacobian matrix into orthogonal matrices to compute its pseudo-inverse:

$$J^{+} = V\Sigma^{+}U^{T} \tag{3.6}$$

This method is beneficial for ill-conditioned or singular matrices because the singular value decomposition of the Jacobian always exists. SVD is used to deduce the Selectively Damped Least Squares method [10].

Damped Least Squares (DLS)

The damped least squares method [10] adds a damping factor to the Jacobian to improve solution stability, expressed as:

$$\Delta q = J^{T}(JJ^{T} + \lambda^{2}I)^{-1}\vec{e} \tag{3.7}$$

This method prevents large joint velocities and can stabilize the solution near singularities. The choice of λ is critical for balancing convergence speed and stability [18].

Selectively Damped Least Squares (SDLS)

The SDLS introduced in [54] method combines SVD and DLS, allowing for selective damping of individual joints. SDLS improves stability and reduces jitter near singularities. SDLS outperforms the Jacobian Transpose and DLS methods in precision, especially for near-singularities, but it may take more iterations to converge.

3.2.2. Heuristic and Meta-Heuristic Methods

Heuristic and meta-heuristic methods use optimization algorithms that iteratively minimize errors between the desired and actual end-effector positions. They are more computationally expensive than Jacobian methods but can handle non-linearities and singularities better. Research showed successful implementations of differential evolutionary algorithms [62] and Particle Swarm Optimization [58]. Additionally, [21] uses Newton methods to solve the IK, while [13] applies a Monte Carlo method. Further common approaches include Style or Mesh-based IK [24, 49], sequential inverse kinematics [52] and the Feedback IK [38]. On the side of heuristic methods, [6] introduced the FABRIK algorithm. The advantage is that since these methods rely on optimization algorithms, they can adapt to new robots and include additional constraints like obstacle avoidance or joint limits.

3.3. Learning-based Approaches

Lastly, we will take a look at learning-based methods. These methods use machine learning algorithms to learn the mapping between the end-effector position p_{eef} and orientation and the robot's joint angles q. They have shown promising results in solving the IK problem for robots with redundant DOF [16, 51].

3.3.1. Regression-Based Approaches

Early work on learning-based IK focused on direct regression techniques. Regression-based approaches provide a direct mapping between the end-effector position and joint configurations. Bocsi et al. used One-Class SVMs to predict feasible configurations in specific workspace regions [9]. Similarly, D'Souza et al. applied locally weighted projection regression to approximate solutions in real-time settings [16].

Neural networks employed by ElSherbiny et al. learn a straightforward mapping but fail to capture multi-modal solutions due to the inherent ambiguity in the IK problem [46]. Csiszar et al. proposed dataset partitioning to reduce that ambiguity, which improved accuracy but at the cost of scalability [15].

3.3.2. Generative Models

Generative models have gained traction for solving IK due to their ability to capture the one-to-many mapping inherent in the problem. Ames et al. introduced IKFlow [3], a framework based on normalizing flows, generating distributions of joint configurations. Ren and Ben-Tzvi employed Generative Adversarial Networks to approximate IK

solutions. However, the generated configurations exhibited the best error of 8 cm, which was worse than a simple fully connected network [41]. [34] proposed a Generative Graphical Inverse Kinematic solver producing multiple diverse solutions with an acceptable error in the mm dimension. Interestingly, that approach can also solve the IK for robots not in the training data.

3.3.3. Graph Neural Networks

Graph-based approaches exploit the kinematic chain structure of robotic arms. Kim et al. utilized Graph Neural Networks (GNNs) to encode redundant robots' null space, allowing efficient mapping of the mapping from task space to joint space [29]. Lehner et al. combined GNN predictions with motion planners, such as RRT, for obstacle-aware IK in constrained environments. These methods excel in structured scenarios but face challenges in scaling with higher degrees of freedom [33, 53].

3.3.4. Bayesian and Variational Methods

Bayesian methods approximate the posterior distribution of IK solutions, modeling uncertainty effectively. [30, 47] applied techniques like Variational Autoencoders (VAEs) and Conditional VAEs (cVAEs) to generate joint distributions conditioned on the end-effector's pose [30, 47]. These methods excel in constrained environments, producing diverse solutions by learning a compact latent representation of feasible configurations.

3.3.5. Hypernetworks and invertible networks

Hypernetworks, which dynamically generate weights for secondary networks based on input conditions, have been employed for modular learning of IK solutions [7]. Galanti and Wolf demonstrated that hypernetworks could enhance flexibility while reducing computational overhead compared to traditional conditioning methods [23]. In [5], an invertible network was presented that can automatically learn the inverse kinematics when learning the forward kinematics.

In this chapter, we have given an overview of the three main methods and looked at the current state of research. In the next chapter, we will introduce our methodology used to solve the IK problem of a robotic arm using reinforcement learning.

4. Method

In this chapter, we will introduce the methodology used to solve the inverse kinematics problem of a robotic arm using reinforcement learning. We will start by defining the problem and the environment in which the robot arm operates. Afterward, we will explain how we generated our datasets. Then, we will introduce the neural network architectures used to solve the problem, followed by their implementations. Finally, we will discuss the optimization techniques used to improve the model's performance.

4.1. Problem Definition

In this thesis, we will let RL algorithms learn the IK for any non-rotational planar two or three DOF robot. We will focus on a one-step environment to immediately receive a solution for the inverse kinematics problem. Therefore, we rewrite the loss function from 2.7 as follows:

$$l = L(s_0, a_0) \tag{4.1}$$

where $a = q$ is the joint configuration of the robot arm predicted by the network, and s_0 is the robot arm encoded with DH parameters and the desired EEF position relative to the base frame.

Before inputting the network the DH parameters, we remove the angle θ from the parameters since we want the network to predict them, and they add no value to the prediction.

Furthermore, if not stated otherwise, we will set the loss function as the Euclidean distance between the predicted and the desired EEF position as defined in 4.2. We calculate the predicted EEF position using the forward kinematics with the predicted joint configuration q and the given DH parameter s_0.

$$L(s_0, a_0) = ||FK(s_0, a_0) - FK(s_0, q)||^2 \tag{4.2}$$

We will consider a solution valid if the loss is below a certain threshold. We will set this threshold to 1 cm.

4.2. Data Generation

We will use a random approach to generate the training data and perform unsupervised learning.

Since we will only be looking at planar robots, we only need to set a within the DH parameters to obtain a valid robot description for each joint. The value ranges between 0.3 m and 0.5 m for a mimicking existing robots.

We will randomly generate values for a and θ to calculate the according EEF position using the forward kinematics. We will then use the calculated EEF position as the desired output for the network and remove θ from the parameters again.

We will generate the data in batches on the fly for training. This way, we can ensure the network does not overfit the training data.

For testing, we will generate a random dataset consisting of 10000 samples. We will use this dataset to evaluate the performance of all the networks. Since we seed the random number generator, we can ensure that all networks' testing sets are equal.

Regarding training data, networks train on the same data if they iterate through the same number of samples. However, an approach with a larger batch size will see more samples and problems than other networks.

4.3. Neural Network Architectures

This section will introduce the classes of neural network architectures used to solve the inverse kinematics problem.

4.3.1. Deterministic Policies

We can divide the deterministic network into the input, hidden, and output layers. We will also refer to the deterministic network as a MLP. To implement the MLP, we used the PyTorch library nn.Module.

Generally, the input layer will consist of the relevant DH parameters of the robot arm and the desired EEF flattened so that one input neuron corresponds to one piece of information. For instance, if we have a two DOF planar robot arm, the adapted DH parameters look like this:

$$s_0 = \begin{bmatrix} 0 & 0.1 & 0 \\ 0 & 0.5 & 0 \end{bmatrix} \tag{4.3}$$

Remember that we omit the angle θ from the parameters. The desired EEF coordinates might look like $(0.2, 0.2)$. The input layer will then consist of 8 neurons.

The hidden layers will consist of fully connected layers with a ReLU activation function. We introduced a hyperparameter to set the size and amount of layers before

the output. The output layer will solely consist of the predicted joint angles of the robot arm.

Angle Prediction

Although directly predicting the joint angles is a valid approach, it is limited as significant output changes in the output can have minimal effects on the EEF position. For instance, when looking at the edge values of the possible angle range $[-\pi, \pi]$. A change in prediction from $-\pi$ to π does not reflect an equivalent change in EEF position.

To overcome this limitation, we will use the atan2 function. As seen in 3.2, the atan2 function takes two arguments and can return the entire angle range. Further, the atan2 function is not restricted in the input range and does not need normalization as a prediction of an angle would need. By taking two numbers as input, the atan2 function will reflect the changes in input more accurately in a change of EEF position. Except for PPO, all networks will use the atan2 function to predict the joint angles. With two input parameters, we will have four neurons in the output layer in our example case. Two of these neurons will represent one joint angle after applying the atan2 function.

4.3.2. Stochastic policies

As a stochastic policy, we will use the stable baselines 3 implementation of PPO. Using this implementation, we only needed to define an environment. To solve the IK, we defined the action space as a continuous space in $[-\pi, \pi]$ for each joint angle. We interpreted the observation for the network agent as the distance between the predicted and the desired EEF position. Therefore, the observation space is also continuous in $[-l_{max}, l_{max}]$. l_{max} is the maximum arm length of the given robot.

After each training step, we reset the environment because we only want a one-step solution to keep it usable as a runtime solution. The resetting triggers the generation of a new problem set.

4.3.3. Recurrent policies

Similarly to the MLP networks, we used the PyTorch library to implement the LSTM networks. As mentioned in 2.3.3, we initialize the hidden and cell state with 0.

We use the same input and output structures as the MLP networks. For the output layer, we set the pytorch parameter *proj_size* to the same number of output neurons used with MLP.

That feature enables us to use a variable hidden state size, as the last hidden state, typically interpreted as the output, is automatically projected to the same output size as the MLP network.

4.3.4. Hybrid policies

We implemented a hybrid network to combine the strengths of the MLP and the LSTM network. The hybrid network uses an LSTM network, whose output is the input for the MLP. The MLP network then predicts the joint angles.

4.4. Implementations

In this section, we will only briefly state the unique characteristics of the implementations. [20] contains the complete code.

4.4.1. Predicting joint angles deterministically

This implementation enables a baseline comparison between direct angle prediction and predicting a distribution for the joint angles. We used a simple MLP network for this category and will refer to this when saying MLP Direct.

4.4.2. Predicting joint angles stochastically

For this category, we implemented the PPO environment from stable baselines 3. For analysis purposes, we replaced the default logger with a custom callback logger to support wandb logging and do custom testing. The custom callback logger introduced a computational overhead, but local testing showed that it is only in the dimension of nanoseconds per step and, therefore, negligible.

We will refer to this implementation as PPO in chapter 5.

4.4.3. Predicting a normal distribution of joint angles using a MLP

We used the same MLP network to predict a normal distribution of joint angles as in the deterministic approach. However, we now predict the mean μ and the standard deviation σ of the normal distribution. For μ, we used the atan2 function as described in 4.3.1. For σ, we first normalized the output to $[0, 1]$ using the sigmoid function. We then multiplied the output by a factor of 0.5 to get a range of $[0, 0.5]$. We limited σ to be at most 0.5 to prevent the network from predicting a too-large deviation, as this also showed increased performance in local testing. To receive an angle, we need to sample

the normal distribution. However, since we need backpropagation to train the network, we cannot use just a random sample, as this would break the gradient. Therefore, we used the reparameterization trick as described below.

We will call this implementation MLP Norm Dist in the following chapter 5.

4.4.4. Reparameterization Trick

The reparameterization trick is a technique used to allow backpropagation through a stochastic node. In our case, the stochastic node is the sampling from a normal distribution. The reparameterization trick embodies sampling from a standard normal distribution and then transforming the sample to the desired distribution. This way the sample $z \sim \mathcal{N}(\mu, \sigma^2)$ becomes:

$$z = \mu + \sigma \cdot \epsilon \tag{4.4}$$

where ϵ is sampled from the standard normal distribution $\epsilon \sim \mathcal{N}(0, 1)$. This way, we can backpropagate through the network and still have the deviation of a distribution. Pytorch has a built-in function to sample from a normal distribution using the reparameterization trick. Nonetheless, we also implemented the reparameterization trick manually to compare it with the built-in function.

4.4.5. Predicting a normal distribution of joint angles using an LSTM

Similarly to the MLP network predicting a distribution, we used an LSTM network to predict the mean μ using atan2 and the standard deviation σ of the normal distribution. To receive a sample, we again used the reparameterization trick.

In chapter 5, we will refer to this implementation as LSTM Norm Dist.

4.4.6. Predicting a beta distribution of joint angles using a MLP

Using a MLP, we now predict each joint's parameters p and q of a beta distribution. We do not normalize the values or map them to a specific range since the beta distribution can handle the entire possible value range of the network. The only modifications we made are to ensure that p and q do not become 0 by adding a $\epsilon = 0.01$ and mapping the sampled result from $[0, 1]$ to the desired range of $[-\pi, \pi]$. To receive a sample from the beta distribution, we used the PyTorch built-in function to sample from a beta distribution using the reparameterization trick.

In chapter 5, we will refer to this implementation as MLP Beta Dist.

4.4.7. Predicting a Gaussian mixed distribution of joint angles using an MLP

This implementation is similar to the normal distribution prediction using an MLP in 4.4.3. The difference is that we now predict the parameters of a Gaussian mixture model. We predict two means μ using atan2 and two standard deviations σ of the normal distributions. We also predict the mixture model's weights a and b. a and b are then normalized using the softmax function to ensure that the weights sum up to 1, representing the probability of the two distributions.

To sample from the mixture model, we first sample one component from the weights using a Bernoulli distribution. Afterward, we again sampled from that component's normal distribution using the reparameterization trick. It is noteworthy that we sample a component for each joint independently.

In 5, we will refer to this implementation as MLP Mixed Gaussian.

4.4.8. Predicting a Gaussian mixed distribution of joint angles using an LSTM + MLP

This implementation is similar to the MLP implementation in 4.4.7. The only difference is that we now use an LSTM and MLP network to predict the parameters of the Gaussian mixture model. Therefore, we automatically feed the LSTM output into the MLP.

In chapter 5, we will refer to this implementation as LSTM + MLP Mixed Gaussian.

4.4.9. Predicting a Gaussian mixed distribution of joint angles using an LSTM

This approach varies from the MLP implementation in 4.4.7 in that we use an LSTM network exclusively. The output of the LSTM is then the parameters for two normal distributions. However, instead of predicting the weights of the mixture model, we let the network predict one additional parameter per robot arm; let's call it α. That output value is normalized using the sigmoid function to ensure that the value is between 0 and 1. We then use that value to predict whether to use the mixture model's first or second component. After selecting the component, we use the same component for all joints. This method enabled the network to learn two distinct correct solutions for the IK problem.

In chapter 5, we will refer to this implementation as LSTM Mixed Gaussian.

In this chapter, we defined the problem, introduced used network architectures, and explained their implementations used in this thesis. In the next chapter, we will discuss the results of the implementations presented above.

5. Experiments

In this chapter, we present the results of the experiments conducted to evaluate the performance of the proposed approaches. We compare the performance of the different models on two and three DOF problems. We present the results in tables and visualizations where they are the most interesting. All visualizations can be found in the appendix A.

5.1. Setup

We ran the tests on a Dell XPS laptop running Ubuntu 24.04.1 LTS with an Intel i7-11800H @ 2.30GHz processor containing 16 cores with 32GB RAM and an Nvidia GeForce RTX 3050 16 GB Laptop GPU. The only exception was the training of the PPO algorithm because of its high computational run times. It was trained on a server computer running Ubuntu 20.04.6 LTS with an Intel Xeon Platinum 8380 @ 2.3 GHz processor containing 160 cores with 32GB RAM and an Nvidia A100-40GB GPU. We should note that we did not use the server exclusively, so we will not state the exact training times, as they are not indicative.

The code was written in Python using the torch and stable baseline 3 libraries. We ran each training 5 times, using the results mean. To monitor and store the results, we used wandb [8].

We trained each implementation for 10000 epochs, except for PPO. We will refer to the time passed during training as training time in the tables 5.1, 5.2, 5.3 and 5.4.

After the training, we created the visualizations and calculated the performance metrics. We define Runtime as the time to predict a single solution after training once we load the model into memory. When we load the model into memory, it costs around 700 µs once. The Batched Runtime is the time it takes to predict a batch of 100 solutions.

DOF	Metric	MLP Direct	PPO
2	Ø error [mm]	**1.24 (1.24, 1.24)**	13.6
	Accuracy [%]	**100.00 (100.00, 100.00)**	69.98
	Training Steps	10'000	50'000'000
	Training Time [s]	**579 (575, 588)**	$3.7 \cdot 10^5$
	# Valid Solutions	1	1
	Avg. Runtime [µs]	**154.44 (154.37, 155.21)**	443.94 (440.22, 445.43)
	Avg. Batched Runtime [µs]	**165.87 (163.89, 165.99)**	470.45 (470.22, 473.84)
3	Ø error [mm]	**1.21 (1.21, 1.21)**	20.40
	Accuracy [%]	**100.00 (100.00, 100.00)**	56.45
	Training Steps	10'000	45'000'000
	Training Time [s]	**658 (644, 679)**	$1.5 \cdot 10^5$
	# Valid Solutions	1	1
	Avg. Runtime [µs]	**157.26 (156.34, 160.01)**	464.74 (461.52, 465.39)
	Avg. Batched Runtime [µs]	**172.20 (172.00, 173.00)**	477.34 (474.90, 481.03)

Table 5.1.: Performance metrics for an MLP network and PPO on two DOF and three DOF tasks. Deviations, if stated, were calculated using bootstrapping. We provide the confidence interval for 95% assurance with 10'000 samples. The batch size of a batched runtime was 100.

5.2. Results

5.2.1. The simplest implementation performs the best for two and three DOF problems

Comparing all results from 5.1, 5.2, 5.3 and 5.4, we can deduce that the MLP network which predicts the angles directly performs the best. It has the lowest mean error and the highest accuracy for two and three DOF problems, which is also visible in 5.1. Combined with the lowest runtime, it is the best option for real-time application out of the compared methods.

A possible explanation for the good performance of the MLP network is that the IK for two and three DOF is not complex enough to benefit from the more complicated models. The problem is simple enough that the MLP network can learn the solution space without needing a more complex model. However, this might not be true for more complicated problems requiring further investigation.

The only drawback of this implementation is that this model always predicts the same solution for the same input. This drawback might be a problem for some applications

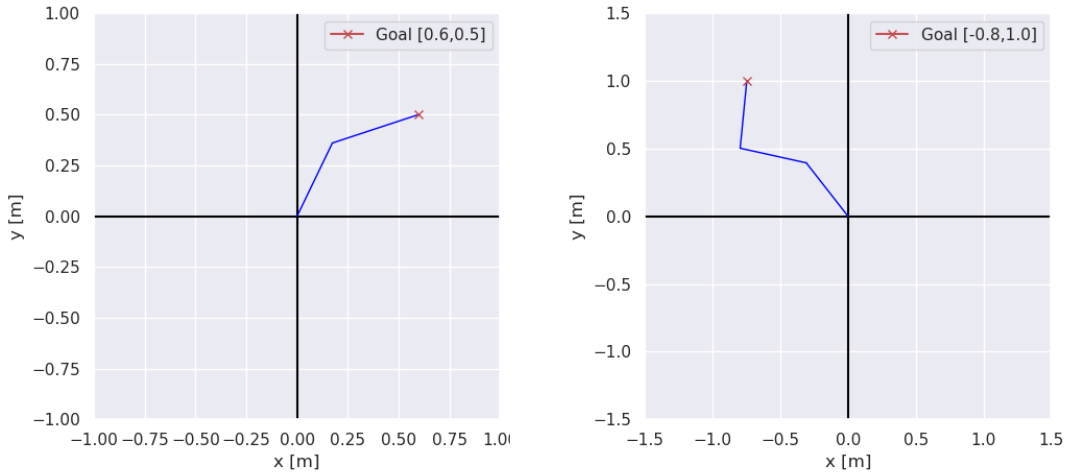


Figure 5.1.: MLP network with two and three DOF predicting the angles directly. The loss of both predictions is 1.1 mm and 1.2 mm, respectively.

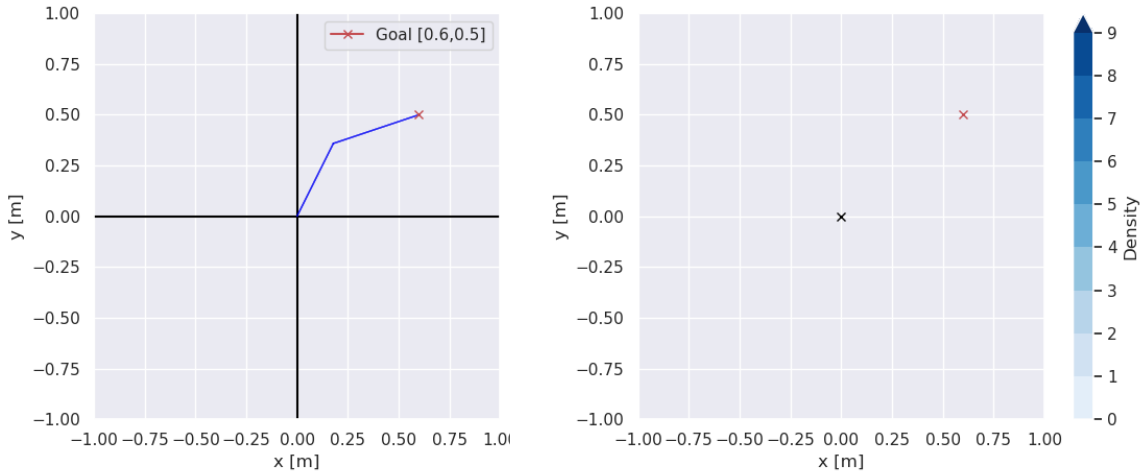


Figure 5.2.: MLP network with two DOF predicting a normal distribution and the corresponding heatmap. The heatmap seems empty due to the high accuracy of the model, but it isn't. The loss of the prediction is averaging at 4.3 mm.

requiring various solutions.

DOF	Metric	MLP Norm Dist	MLP Beta Dist
2	Ø error [mm]	**4.39 (4.39, 4.39)**	58.62 (58.28, 58.87)
	Accuracy [%]	**97.50 (97.50, 97.50)**	5.19 (5.09, 5.44)
	Training Steps	10'000	10'000
	Training Time [s]	**378 (377, 380)**	344 (339, 347)
	# Valid Solutions	1 [1]	1 [2]
	Avg. Runtime [µs]	321.35 (320.01, 322.76)	**207.08 (206.00, 208.16)**
	Avg. Batched Runtime [µs]	446.30 (445.24, 447.60)	**211.27 (210.57, 212.54)**
3	Ø error [mm]	**8.49 (8.49, 8.49)**	69.52 (69.21, 69.82)
	Accuracy [%]	**75.59 (75.59, 75.60)**	8.48 (8.06, 8.73)
	Training Steps	10'000	10'000
	Training Time [s]	**452 (450, 454)**	426 (425, 428)
	# Valid Solutions	1 [1]	1 [2]
	Avg. Runtime [µs]	372.48 (371.12, 373.96)	**256.77 (255.50, 258.04)**
	Avg. Batched Runtime [µs]	493.68 (492.00, 495.36)	**268.62 (267.00, 270.24)**

Table 5.2.: Performance metrics for an MLP network predicting a normal and beta distribution respectively on two DOF and three DOF tasks. Deviations, if stated, were calculated using bootstrapping. We provide the confidence interval for 95% assurance with 10'000 samples. The batch size of a batched runtime was 100. [1]*The network outputs a normal distribution of one solution.* [2]*The network outputs a beta distribution.*

5.2.2. MLP and LSTM networks perform better and faster than PPO

A second finding is that the MLP and LSTM networks perform better and faster than the PPO algorithm. The training time is also significantly higher than the training time of the MLP and LSTM networks.

The PPO algorithm, initially designed for a multi-step problem where the agent needs to learn a sequence of actions to solve the problem, may cause worse performance. In contrast, we created the IK in our case as a single-step problem, where the agent has to learn a single action to solve the problem. Further, even though PPO trains the value network correctly 5.5, the value loss is unnecessary for a single-step problem and, therefore, just a computational overhead.

DOF	Metric	LSTM Norm Dist	LSTM Mixed Gaussian
2	Ø error [mm]	**2.13 (2.13, 2.13)**	9.25 (9.24, 9.26)
	Accuracy [%]	**99.96 (99.96, 99.96)**	67.71 (67.54, 67.83)
	Training Steps	10'000	10'000
	Training Time [s]	$1.01 \cdot 10^3$ ($1.01 \cdot 10^3$, $1.02 \cdot 10^3$)	**302 (301, 303)**
	# Valid Solutions	1 [1]	2 [2]
	Avg. Runtime [µs]	474.33 (473.11, 475.66)	**366.23 (365.89, 367.46)**
	Avg. Batched Runtime [µs]	839.70 (838.00, 841.40)	**408.12 (407.08, 409.24)**
3	Ø error [mm]	5.44 (5.44, 5.44)	**4.05 (4.03, 4.07)**
	Accuracy [%]	**93.25 (93.24, 93.26)**	92.99 (92.91, 93.10)
	Training Steps	10'000	10'000
	Training Time [s]	$1.02 \cdot 10^3$ ($1.01 \cdot 10^3$, $1.02 \cdot 10^3$)	**967 (962, 972)**
	# Valid Solutions	1 [1]	2 [2]
	Avg. Runtime [µs]	525.65 (524.52, 527.30)	**514.48 (513.55, 515.96)**
	Avg. Batched Runtime [µs]	924.78 (923.76, 926.56)	**585.03 (583.00, 587.06)**

Table 5.3.: Performance metrics for an LSTM network predicting a normal distribution and a Mixed Gaussian distribution on two DOF and three DOF tasks. Deviations, if stated, were calculated using bootstrapping. We provide the confidence interval for 95% assurance with 10'000 samples. The batch size of a batched runtime was 100. [1]*The network outputs a normal distribution of one solution.* [2]*The network outputs a mixed Gaussian distribution for two distinct solutions.*

5.2.3. Non-trivial connection between complexity and performance

Third, the results reflect a non-trivial connection between the complexity of the model and the performance. For some implementations, a higher network complexity performs better.

We can see this finding by comparing the MLP network predicting a normal distribution with the LSTM network predicting a normal distribution, as reflected in Table 5.2 and Table 5.3.

On the contrary, when combining the LSTM and MLP network to predict a mixed Gaussian distribution, the performance worsens with the increased complexity of the model. This relation is reflected in Table 5.4. Similarly, the increased complexity in PPO does not lead to better performance, as discussed in 5.2.2.

We conclude, therefore, that the computational overhead will rarely be worth the marginal gains in performance.

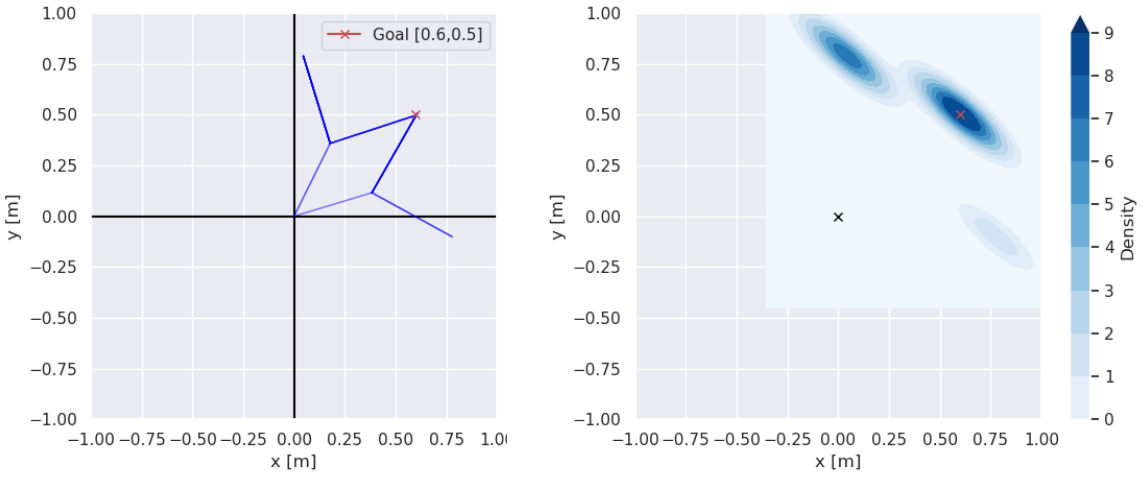


Figure 5.3.: LSTM network with two DOF predicting a Mixed Gaussian distribution and the corresponding heatmap. The loss of the prediction is, on average, 5.5 mm.

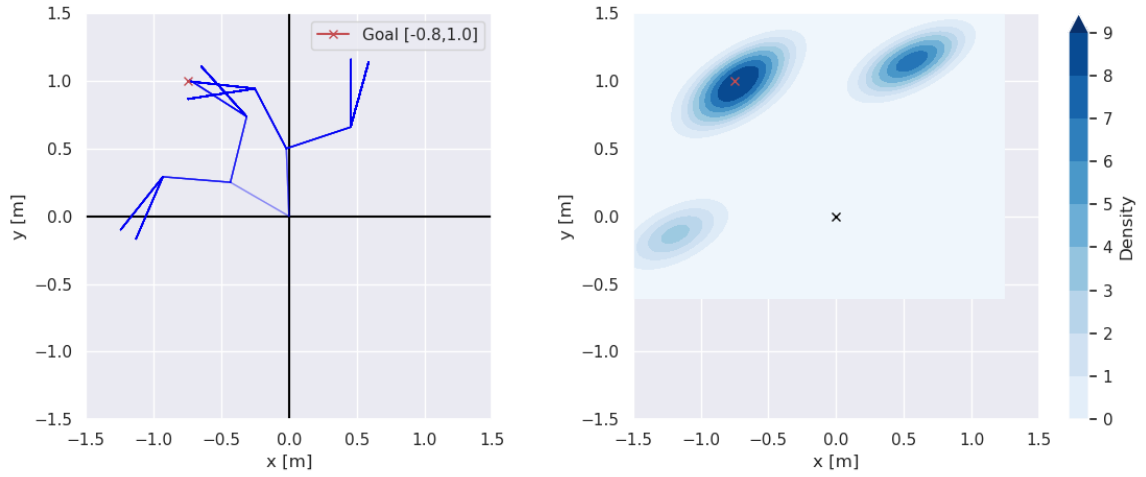


Figure 5.4.: LSTM network with three DOF predicting a Mixed Gaussian distribution and the corresponding heatmap. The average loss of the prediction is 11.9 mm.

DOF	Metric	MLP Mixed Gaussian	LSTM+MLP Mixed Gaussian
2	Ø error [mm]	**1.95 (1.95, 1.95)**	5.51 (5.51, 5.51)
	Accuracy [%]	**99.94 (99.93, 99.94)**	91.21 (90.95, 91.32)
	Training Steps	10'000	10'000
	Training Time [s]	650 (644, 653)	**248 (247, 250)**
	# Valid Solutions	1 [1]	1 [1]
	Avg. Runtime [µs]	**269.49 (268.87, 271.25)**	758.57 (757.33, 760.49)
	Avg. Batched Runtime [µs]	**309.14 (308.13, 310.57)**	1018.77 (1017.82, 1020.54)
3	Ø error [mm]	**3.00 (3.00, 3.00)**	11.98 (11.87, 12.11)
	Accuracy [%]	**99.72 (99.70, 99.73)**	67.63 (67.50, 67.74)
	Training Steps	10'000	10'000
	Training Time [s]	853 (848, 858)	**624 (620, 626)**
	# Valid Solutions	1 [1]	1 [1]
	Avg. Runtime [µs]	**336.67 (335.63, 338.34)**	862.65 (861.67, 864.30)
	Avg. Batched Runtime [µs]	**397.54 (396.11, 399.08)**	1119.36 (1118.24, 1120.72)

Table 5.4.: Performance metrics for an MLP and a combination of LSTM and MLP network predicting a Mixed Gaussian distribution on two DOF and three DOF tasks. Deviations, if stated, were calculated using bootstrapping. We provide the confidence interval for 95% assurance with 10'000 samples. The batch size of a batched runtime was 100. [1]*The network outputs a mixed Gaussian distribution of one solution.*

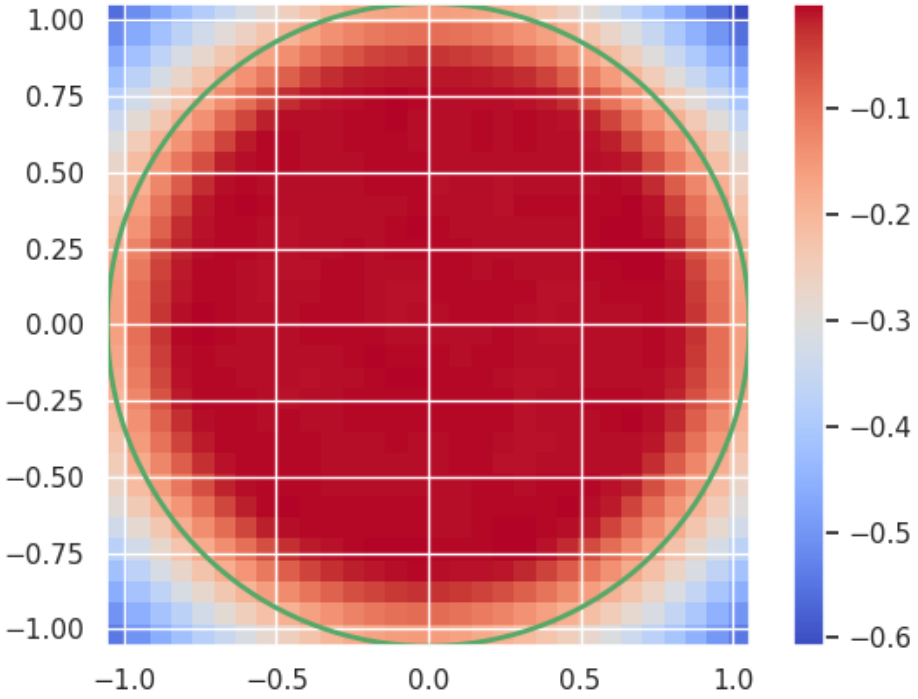


Figure 5.5.: Value loss function evaluated at every (x,y) coordinates pair with a trained PPO network. The green circle indicates the maximum arm length from the center

6. Conclusion and Outlook

6.1. Conclusion

In this thesis, we gave an overview of the IK problem and the different approaches to solving it. Afterward, we presented the current most comment state of the art approaches in solving the IK for robots. Next, we presented our RL approaches consisting of the MLP, LSTM, and PPO networks. We analyzed the performances when adapting to what the networks predicted. We compared predicting angles directly and via a normal, beta, and mixed Gaussian distribution. Our key findings are that the most straightforward approach, predicting the angles directly with MLP, is the most reliable and stable approach. Secondly, we found that using PPO to solve the IK performs worse than the other approaches and takes longer to train. Lastly, we concluded that there is a non-trivial relationship between the network's complexity and the model's performance.

Therefore, we recommend starting roboticists to use the MLP network to solve the IK problem because it had the best performance. We should avoid using the PPO network when training in a single-step environment and remember that higher network complexity does not necessarily lead to better performance.

6.2. Outlook

There are several directions for future work. First, we can extend the proposed approach to solve the IK problem for robots with more complex kinematic structures, such as humanoid robots or robots with non-standard joint types in 3D spaces and higher DOF.

Second, we can explore adding additional constraints to the IK problem, such as joint limits, collision avoidance, or task-specific constraints.

Lastly, further investigation of the mixed Gaussian distribution approach would be intriguing. The first step would be to increase the number of components in the mixture and evaluate the impact on the model's performance. Additionally, we can explore other alternatives for a distribution, such as a composition of trigonometric functions or a mixture of different distributions.

A. Appendix

A.1. Visualization

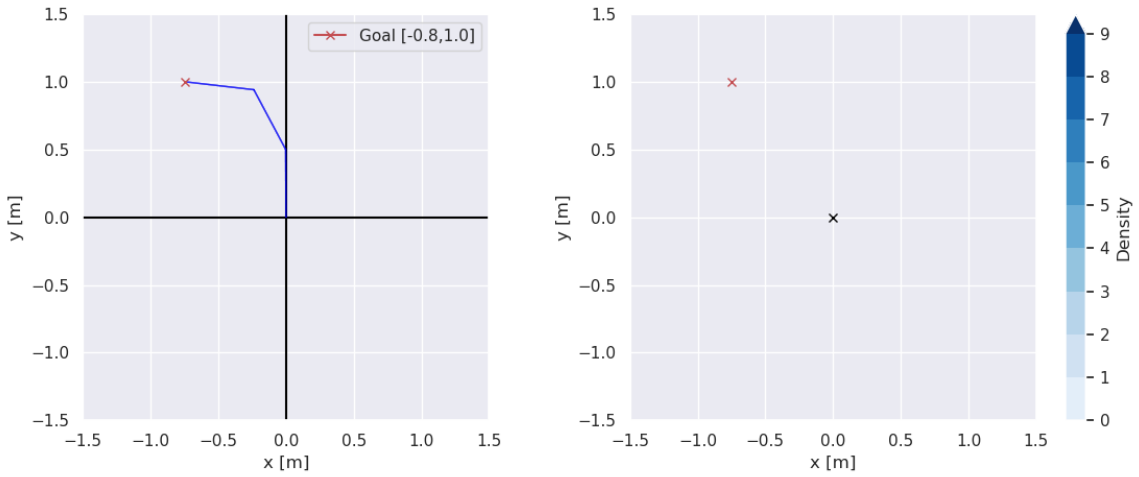


Figure A.1.: Three DOF MLP Dist prediction and the corresponding heatmap. The heatmap seems empty due to the high accuracy of the model, but isn't.

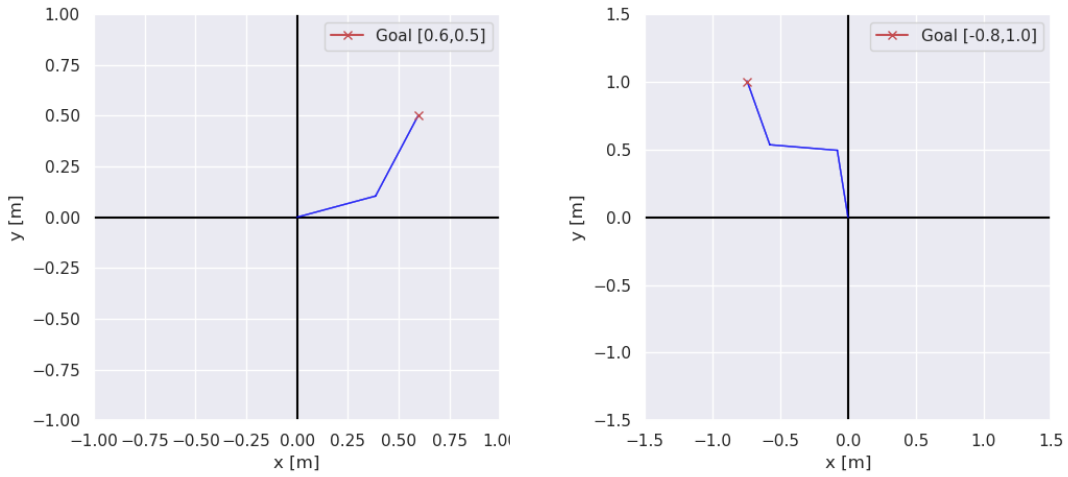


Figure A.2.: Two and three DOF LSTM distribution prediction. A heatmap is for either one not useful as it is an empty plot comparable with A.2

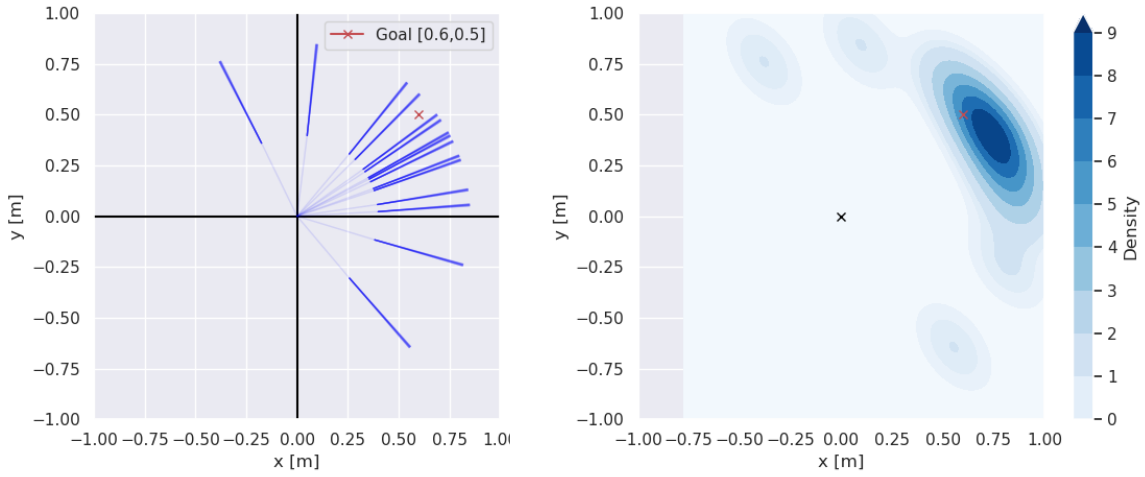


Figure A.3.: Two DOF Beta distribution prediction and the corresponding heatmap.

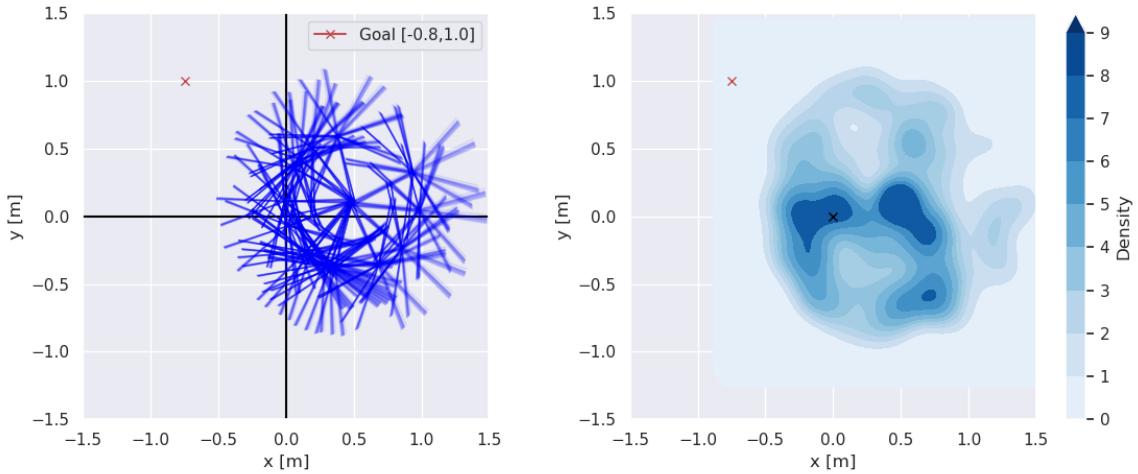


Figure A.4.: Three DOF MLP Beta distribution prediction and the corresponding heatmap.

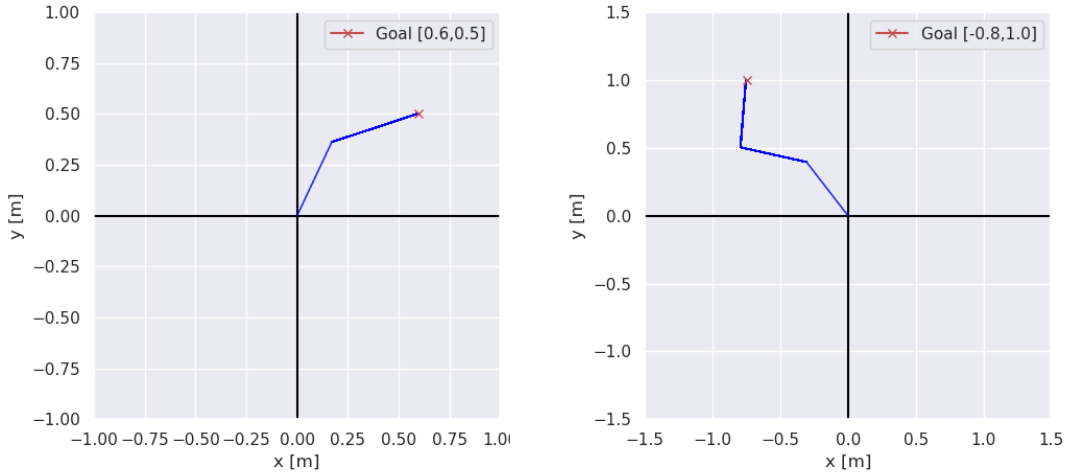


Figure A.5.: Two and three DOF MLP Network predicting a mixed Gaussian distribution. A heatmap is for either one not useful as it as an empty plot comparable with A.2

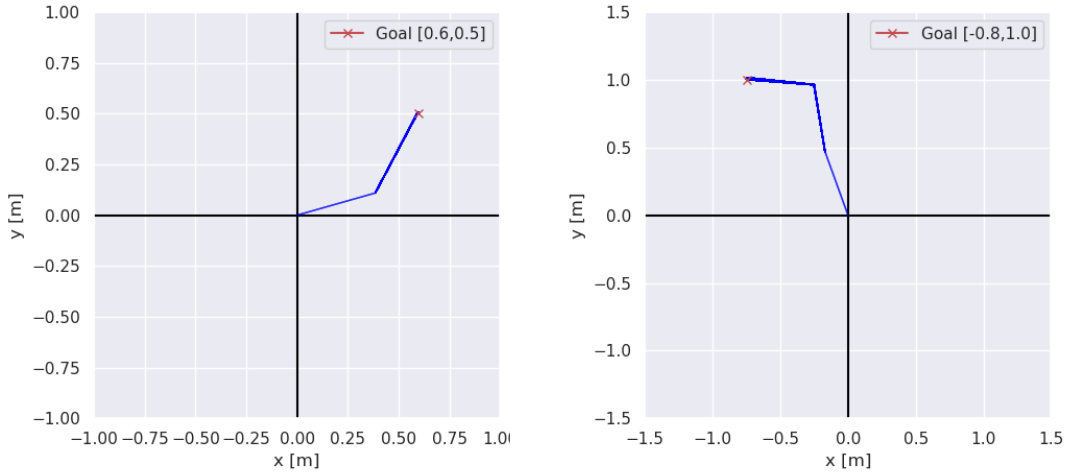


Figure A.6.: Two and three DOF Hybrid Network consisting of an LSTM and MLP network predicting a mixed Gaussian distribution. A heatmap is for either one not useful as it is an empty plot comparable with A.2

Abbreviations

FK Forward Kinematics

IK Inverse Kinematics

DOF degrees of freedom

MLP multilayer perceptron

RL Reinforcement Learning

PPO Proximal Policy Optimization

LSTM Long Short-Term Memory

EEF End Effector

Bibliography

[1] T. M. Abdel-Rahman. "A Generalized Practical Method for Analytic Solution of the Constrained Inverse Kinematics Problem of Redundant Manipulators." In: *The International Journal of Robotics Research* 10.4 (1991), pp. 382–395.

[2] J. Achiam. "Spinning up in Deep Reinforcement Learning." In: (2018).

[3] B. Ames, J. Morgan, and G. Konidaris. "IKFlow: Generating Diverse Inverse Kinematics Solutions." In: *IEEE Robotics and Automation Letters* 7.3 (2022), pp. 7177–7184.

[4] J. Angeles. "On the Numerical Solution of the Inverse Kinematic Problem." In: *The International Journal of Robotics Research* 4.2 (1985), pp. 21–37.

[5] L. Ardizzone, J. Kruse, C. Rother, and U. Köthe. "Analyzing Inverse Problems with Invertible Neural Networks." In: *Proc. of the Int. Conf. on Learning Representations (ICLR)*. 2019.

[6] A. Aristidou and J. Lasenby. "FABRIK: A Fast, Iterative Solver for the Inverse Kinematics Problem." In: *Graphical Models* 73.5 (2011), pp. 243–260.

[7] R. Bensadoun, S. Gur, N. Blau, and L. Wolf. "Neural Inverse Kinematic." In: *Proc. of the Int. Conf. on Machine Learning (ICML)*. Ed. by K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato. Vol. 162. 2022, pp. 1787–1797.

[8] L. Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020.

[9] B. Bocsi, D. Nguyen-Tuong, L. Csato, B. Scholkopf, and J. Peters. "Learning Inverse Kinematics with Structured Prediction." In: *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. 2011, pp. 698–703.

[10] S. R. Buss. "Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods." In: *IEEE Journal of Robotics and Automation* 17.1-19 (2004).

[11] I.-M. Chen and Y. Gao. "Closed-form inverse kinematics solver for reconfigurable robots." In: *Proc. of the IEEE Int. Conf. on Robotics and Automation (ICRA)*. Vol. 3. 2001, pp. 2395–2400.

[12] S. Chiaverini. "Redundant Robots." In: *Encyclopedia of Systems and Control*. Ed. by J. Baillieul and T. Samad. 2020, pp. 1–10.

[13] K. W. Chin, B. von Konsky, and A. Marriott. "Closed-form and generalized inverse kinematics solutions for the analysis of human motion." In: *Int. Conf. of the IEEE Engineering in Medicine and Biology Society (EMBC)*. Vol. 5. 1997, pp. 1911–1914.

[14] G. Chirikjian and J. Burdick. "Hyper-redundant robot mechanisms and their applications." In: *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. Vol. 1. 1991, pp. 185–190.

[15] A. Csiszar, J. Eilers, and A. Verl. "On Solving the Inverse Kinematics Problem Using Neural Networks." In: *Proc. of the IEEE Int. Conf. on Mechatronics and Machine Vision in Practice (M2VIP)*. 2017, pp. 1–6.

[16] A. D'Souza, S. Vijayakumar, and S. Schaal. "Learning Inverse Kinematics." In: *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*. Vol. 1. 2001, pp. 298–303.

[17] J. Denavit and R. S. Hartenberg. "A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices." In: *Journal of Applied Mechanics* 22.2 (1955), pp. 215–221.

[18] A. Deo and I. Walker. "Robot Subtask Performance with Singularity Robustness Using Optimal Damped Least-Squares." In: *Proc. of the IEEE Int. Conf. on Robotics and Automation (ICRA)*. 1992, pp. 434–441.

[19] R. Diankov. "Automated Construction of Robotic Manipulation Programs." 2010.

[20] M. A. Emanuel Andre. *Bachelor Thesis Repository*. Version 1.0.0. 2024.

[21] R. Fletcher. *Practical Methods of Optimization*. 2nd ed. John Wiley & Sons, 2000.

[22] S. Fountas, N. Mylonas, I. Malounas, E. Rodias, C. Hellmann Santos, and E. Pekkeriet. "Agricultural Robotics for Field Operations." In: *Sensors* 20.9 (2020).

[23] T. Galanti and L. Wolf. "On the Modularity of Hypernetworks." In: *Proc. of the Int. Conf. on Neural Information Processing Systems (NeurIPS)*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin. Vol. 33. 2020, pp. 10409–10419.

[24] K. Grochow, S. L. Martin, A. Hertzmann, and Z. Popović. "Style-based inverse kinematics." In: *ACM Trans. Graph.* 23.3 (2004), pp. 522–531.

[25] L. Herrera-Bendezu, E. Mu, and J. Cain. "Symbolic Computation of Robot Manipulator Kinematics." In: *Proc. of the IEEE Int. Conf. on Robotics and Automation (ICRA)*. 1988, pp. 993–998.

[26] S. Hochreiter and J. Schmidhuber. "Long Short-Term Memory." In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[27] R. D. Howe and Y. Matsuoka. "Robotics for Surgery." In: *Annual Review of Biomedical Engineering* 1.1 (1999), pp. 211–240.

[28] M. Javaid, A. Haleem, R. P. Singh, and R. Suman. "Substantial Capabilities of Robotics in Enhancing Industry 4.0 Implementation." In: *Cognitive Robotics* 1 (2021), pp. 58–75.

[29] J. T. Kim, J. Park, S. Choi, and S. Ha. "Learning Robot Structure and Motion Embeddings Using Graph Neural Networks." In: (2021).

[30] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling. "Improved Variational Inference with Inverse Autoregressive Flow." In: *Proc. of the Int. Conf. on Neural Information Processing Systems (NeurIPS)*. Ed. by D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett. Vol. 29. 2016.

[31] M. Kyrarini, F. Lygerakis, A. Rajavenkatanarayanan, C. Sevastopoulos, H. R. Nambiappan, K. K. Chaitanya, A. R. Babu, J. Mathew, and F. Makedon. "A Survey of Robots in Healthcare." In: *Technologies* 9.1 (2021).

[32] T. Lane. "A Short History of Robotic Surgery." In: *The Annals of The Royal College of Surgeons of England* 100.6_sup (2018), pp. 5–7.

[33] P. Lehner, M. A. Roa, and A. Albu-Schaffer. "Kinematic Transfer Learning of Sampling Distributions for Manipulator Motion Planning." In: *Proc. of the IEEE Int. Conf. on Robotics and Automation (ICRA)*. 2022, pp. 7211–7217.

[34] O. Limoyo, F. Marić, M. Giamou, P. Alexson, I. Petrović, and J. Kelly. *Generative Graphical Inverse Kinematics*. 2024.

[35] *A Note on Denavit-Hartenberg Notation in Robotics*. Vol. Volume 7: 29th Mechanisms and Robotics Conference, Parts A and B. 2005, pp. 921–926.

[36] F. Merat. "Introduction to Robotics: Mechanics and Control." In: *IEEE Journal on Robotics and Automation* 3.2 (1987), pp. 166–166.

[37] B. E. Paden. "Kinematics and Control of Robot Manipulators." PhD thesis. 1986.

[38] A. N. Pechev. "Inverse Kinematics without Matrix Inversion." In: *Proc. of the IEEE Int. Conf. on Robotics and Automation (ICRA)*. 2008, pp. 2005–2012.

[39] D. L. Peiper. "The Kinematics of Manipulators under Computer Control." In: 1968.

[40] M. Raghavan and B. Roth. "Inverse Kinematics of the General 6R Manipulator and Related Linkages." In: *Journal of Mechanical Design* 115.3 (1993), pp. 502–508.

[41] H. Ren and P. Ben-Tzvi. "Learning Inverse Kinematics and Dynamics of a Robotic Manipulator Using Generative Adversarial Networks." In: *Robotics and Autonomous Systems* 124 (2020).

[42] Y. Rizk, M. Awad, and E. W. Tunstel. "Cooperative Heterogeneous Multi-Robot Systems: A Survey." In: *ACM Computing Surveys* 52.2 (2020), pp. 1–31.

[43] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. *Proximal Policy Optimization Algorithms*. 2017.

[44] F. Semeraro, A. Griffiths, and A. Cangelosi. "Human–Robot Collaboration and Machine Learning: A Systematic Review of Recent Research." In: *Robotics and Computer-Integrated Manufacturing* 79 (2023).

[45] W. Shanda, L. Xiao, L. Qingsheng, and H. Baoling. "Existence Conditions and General Solutions of Closed-form Inverse Kinematics for Revolute Serial Robots." In: *Applied Sciences* 9.20 (2019).

[46] A. El-Sherbiny, M. A. Elhosseini, and A. Y. Haikal. "A Comparative Study of Soft Computing Methods to Solve Inverse Kinematics Problem." In: *Ain Shams Engineering Journal* 9.4 (2018), pp. 2535–2548.

[47] K. Sohn, H. Lee, and X. Yan. "Learning Structured Output Representation Using Deep Conditional Generative Models." In: *Proc. of the Int. Conf. on Neural Information Processing Systems (NeurIPS)*. Ed. by C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett. Vol. 28. 2015.

[48] *Springer Handbook of Robotics*. 2nd edition. Springer Berlin Heidelberg, 2016.

[49] R. W. Sumner, M. Zwicker, C. Gotsman, and J. Popović. "Mesh-Based Inverse Kinematics." In: *ACM Transactions on Graphics* 24.3 (2005), pp. 488–495.

[50] C. Szepesvári. *Algorithms for Reinforcement Learning*. Springer nature, 2022.

[51] J. Tenhumberg, A. Mielke, and B. Bäuml. "Efficient Learning of Fast Inverse Kinematics with Collision Avoidance." In: *Proc. of the IEEE-RAS Int. Conf. on Humanoid Robots (Humanoids)*. 2023, pp. 1–8.

[52] L. Unzueta, M. Peinado, R. Boulic, and Á. Suescun. "Full-Body Performance Animation with Sequential Inverse Kinematics." In: *Graphical Models* 70.5 (2008), pp. 87–104.

[53] J. Walter and H. Ritter. "Rapid Learning with Parametrized Self-Organizing Maps." In: *Neurocomputing* 12.2-3 (1996), pp. 131–153.

[54] C. Wampler. "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods." In: *IEEE Transactions on Systems, Man, and Cybernetics* 16.1 (1986), pp. 93–101.

[55] L.-C. Wang and C. Chen. "A Combined Optimization Method for Solving the Inverse Kinematics Problems of Mechanical Manipulators." In: *IEEE Transactions on Robotics and Automation* 7.4 (1991), pp. 489–499.

[56] M. Wiering and M. Van Otterlo, eds. *Reinforcement Learning: State-of-the-Art*. Vol. 12. Springer Berlin Heidelberg, 2012.

[57] W. Wolovich and H. Elliott. "A Computational Technique for Inverse Kinematics." In: *Proc. of the IEEE Conf. on Decision and Control (CDC)*. 1984, pp. 1359–1363.

[58] L. Yiyang, J. Xi, B. Hongfei, W. Zhining, and S. Liangliang. "A General Robot Inverse Kinematics Solution Method Based on Improved PSO Algorithm." In: *IEEE Access* 9 (2021), pp. 32341–32350.

[59] T. Yue-sheng and X. Ai-ping. "Extension of the Second Paden-Kahan Sub-problem and Its' Application in the Inverse Kinematics of a Manipulator." In: *Proc. of the IEEE Conf. on Robotics, Automation and Mechatronics (RAM)*. 2008, pp. 379–381.

[60] I. Zaplana, H. Hadfield, and J. Lasenby. "Closed-Form Solutions for the Inverse Kinematics of Serial Robots Using Conformal Geometric Algebra." In: *Mechanism and Machine Theory* 173 (2022).

[61] D. Zhang and B. Hannaford. "IKBT: Solving Symbolic Inverse Kinematics with Behavior Tree, wobei man das ruhig auch klein schreiben kann." In: *Journal of Artificial Intelligence Research* 65 (2019), pp. 457–486.

[62] Q. Zhang, D. Wang, and L. Gao. "Research on the Inverse Kinematics of Manipulator Using an Improved Self-Adaptive Mutation Differential Evolution Algorithm." In: *International Journal of Advanced Robotic Systems* 18.3 (2021).